CONTENTS

Intermediate | AI agents and LLM context engineering | IBM Technology

MCP vs Skills: Two Ways to Extend an LLM's Context

Understand when to wire an LLM to live external data with MCP, and when to teach it a repeatable procedure with a skill.

For developers building AI agents or coding assistants who need to decide how to extend an LLM beyond its training data · 27 July 2026

Watch the source video | Read the condensed version | Read the highlights

BEFORE YOU START

- Basic familiarity with what an LLM is and how a context window works
- Basic familiarity with what an API call and an access token are

Overview

A large language model only knows what it saw during training, plus whatever text you put in its context window at request time. Getting a useful answer out of it is less about the wording of your question and more about what surrounding information you supply — this is context engineering, and it is broader than prompt engineering. Two complementary mechanisms have emerged for supplying that context at scale: the Model Context Protocol (MCP) and skills.

MCP standardizes how an LLM-based application talks to external systems — CRMs, databases, internal APIs — so the model can fetch or act on live data through a scoped, authenticated interface instead of you hand-rolling API glue for every tool. Skills solve a different problem: they package a reusable, repeatable procedure — a way of doing a task the same way every time — as a small bundle of instructions (and optionally scripts and resources) that the model loads into its context only when the task calls for it.

This lesson explains both mechanisms, shows the shape of a real MCP tool call and a real skill file, and gives you a rule of thumb for choosing between them or using both together.

KEY TAKEAWAYS

- ✓ An LLM answers well only when its context window contains the right information, not just a well-phrased question — this is the difference between prompt engineering and context engineering.
- ✓ MCP (Model Context Protocol) standardizes how an LLM-based application calls external services: it turns a service's API into an LLM-readable set of tools, and it handles scoped, authenticated access to that service.
- ✓ Without MCP, giving a model raw API documentation and a token and hoping it calls the API correctly is fragile and unauditable.
- ✓ MCP does not make an LLM's behavior repeatable — LLMs are non-deterministic, so even with the same data source, output formatting and process can vary run to run.
- ✓ A skill is a packaged, reusable set of instructions (typically a markdown file plus metadata, optionally with scripts and resources) that teaches the model to perform a task the same way every time.
- ✓ Skills are auto-loaded into the context window only when relevant, so they don't bloat every request with instructions the current task doesn't need.
- ✓ Choose MCP when the agent needs controlled, permissioned access to real-time external data. Choose a skill when you need a lightweight, repeatable way of doing something the model doesn't already know how to do consistently.
- ✓ MCP and skills are complementary, not competing: an agent can use MCP to fetch live CRM data and a skill to guarantee that data is always formatted and processed the same way.

01 From prompt engineering to context engineering

0:00

> An LLM is a pattern-matching system trained on huge amounts of text; the quality of its answer depends on what extra information you give it beyond the question itself.

A large language model is trained on a broad slice of publicly available text — books, articles, forum threads, documentation — and learns statistical patterns over that text. That lets it answer a very wide range of questions reasonably well, but it has no built-in knowledge of your specific database schema, your team's formatting conventions, or data that changed after training ended.

Asking a question with a role and a task attached — "you are a database assistant, answer this question about our schema" — is prompt engineering. It shapes how the model responds, but it doesn't give the model anything it didn't already have. Context engineering goes further: it is the discipline of deciding what additional information — reference data, formatting rules, live tool output, prior conversation state — needs to sit inside the context window for the model to produce a correct, useful answer. The model's output quality is bounded by the quality and relevance of what's in that window, not just by how the question is phrased.

This distinction matters because it reframes the engineering problem. Instead of asking "how do I word this prompt better," you ask "what information does the model need to see, and how do I get it there reliably, automatically, and only when relevant." MCP and skills are two different, complementary answers to that second question.

KEY POINTS

- → An LLM's answer quality depends on what's in its context window, not only on how the question is phrased.
- → Prompt engineering shapes the instruction; context engineering supplies the information the instruction needs to be answered correctly.
- → Context can include reference data, formatting conventions, and live tool output — anything the model needs but wasn't trained on.
- → The core engineering question becomes: what does the model need to see, and how do I deliver it reliably and only when needed?

Prompt engineering vs context engineering, side by side

The first prompt only tells the model what to do. The second gives it the concrete information required to do it correctly — that second version is context engineering, even though it's still "just" a prompt.

```
# Prompt engineering only
prompt_v1 = "What is the contact info for customer Acme Corp?"

# Context engineering: the answer-relevant data is placed in the window
prompt_v2 = f"""
You are a CRM assistant. Use only the record below to answer.

Customer record:
{{
  "name": "Acme Corp",
  "contact_email": "ops@acme.example",
  "contact_phone": "+1-555-0199",
  "account_owner": "J. Rivera"
}}

Question: What is the contact info for customer Acme Corp?
"""
```

02 MCP: a standard interface between an LLM and external services

2:55

> The Model Context Protocol standardizes how an AI application calls external data sources and services, so the model doesn't need raw API docs and a bare token to get live data.

Suppose an agent needs to read or update a record in a CRM. The naive approach is to paste the CRM's full API documentation and an access token into the model's context and instruct it to make the right calls without mistakes. That's fragile: the model has to correctly interpret arbitrary API docs, form valid requests, and handle authentication, all inside free-form text generation, with no guarantee it gets any of that right.

MCP replaces that with a standard protocol between an AI application (an IDE, an agent runtime) and an MCP server that sits in front of a specific service. The MCP server exposes a small set of well-defined, LLM-readable tools — for example "get_customer" or "update_contact" — each with a typed schema for its inputs and outputs. The model doesn't write raw HTTP; it emits a structured request for a named tool with specific arguments. The MCP server is the thing that translates that structured request into the underlying service's actual POST or GET calls, and it's also the place authentication lives: a uniquely scoped token with exactly the read or write access the agent needs, rather than a broad credential passed straight to the model.

Because this interface is standardized, it's supported broadly across AI tools — an MCP server written once for a CRM can be plugged into any MCP-compatible client. MCP solves the problem of connecting a model to external, live data sources in a controlled way. It does not, by itself, make the model behave the same way every time it uses that data — that's a separate problem, covered next.

KEY POINTS

- → MCP standardizes how an AI application talks to external data sources and services, instead of relying on the model to interpret raw API docs.
- → An MCP server exposes typed, LLM-readable tools; the model requests a tool by name and arguments rather than writing raw HTTP calls.
- → The MCP server translates the model's structured tool request into the service's actual GET/POST requests.
- → Authentication and access scope live in the MCP server, so the agent gets a uniquely scoped, read/write-limited token instead of a raw credential.
- → Because MCP is a standard, one server can be reused across different MCP-compatible AI applications.

A minimal MCP-style tool call over JSON-RPC

MCP uses JSON-RPC 2.0 messages. The client (the AI application) first discovers what tools a server offers, then calls one with structured arguments instead of the model writing a raw HTTP request itself.

```
// 1. Client asks the MCP server what tools it exposes
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/list"
}

// 2. Server responds with a typed tool definition
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "tools": [
      {
        "name": "get_customer",
        "description": "Fetch a customer record by name from the CRM",
        "inputSchema": {
          "type": "object",
          "properties": { "customer_name": { "type": "string" } },
          "required": ["customer_name"]
        }
      }
    ]
  }
}

// 3. Client (on the model's behalf) calls the tool with arguments
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "tools/call",
  "params": {
    "name": "get_customer",
    "arguments": { "customer_name": "Acme Corp" }
  }
}

// 4. Server translates this into a real CRM API GET request behind the
// scenes, using its own scoped token, and returns structured content
// back into the model's context window.
```

03 What MCP doesn't solve: repeatable, non-deterministic behavior

3:39

> LLMs are non-deterministic, so even with reliable access to live data via MCP, the model can format or process that data differently each time unless it's told a fixed procedure to follow.

MCP guarantees that the model can reach the right data source with the right permissions. It says nothing about what the model then does with that data. LLMs generate text probabilistically, so two runs of "fetch this customer and summarize it" can come back formatted differently, in a different order, or with different fields emphasized — even against the exact same underlying record.

That's a problem the moment a task needs to be done the same way every time. A sales team pulling CRM data might need the output to always include the customer's name, contact information, and a specific extra field their process depends on, in a fixed structure, every single time, regardless of who on the team is asking or how they phrased the request. Passing the model better prompts each time is not a durable fix, because prompting is not a mechanism for enforcing repeatability — it's advisory, and the model is free to interpret it differently across runs.

What's actually needed is a way to give the model domain knowledge — "here is exactly how we do this task" — that gets applied consistently and is loaded automatically whenever that specific task comes up, without the user having to re-explain it or paste it in by hand each time. That's the gap skills are built to fill.

KEY POINTS

- → MCP solves data access, not output consistency — the model can still produce different results across runs from the same data.
- → LLM non-determinism means repeated instructions in a prompt are advisory, not a guarantee of consistent behavior.
- → A repeatable business process (fixed fields, fixed structure, every time) needs a mechanism stronger than "ask nicely" in each prompt.
- → The missing piece is a way to encode domain-specific, repeatable procedure and have it applied automatically when relevant.

Same data, two different outputs

Given the identical CRM record, an LLM asked twice with only a loose instruction can legitimately produce two structurally different summaries, because nothing pins the output format.

```
Run 1 output:
"Acme Corp — reach them at ops@acme.example or +1-555-0199. Account owner: J. Rivera."

Run 2 output (same record, same loose prompt):
"Customer: Acme Corp\nOwner: J. Rivera\nPhone: +1-555-0199\nEmail: ops@acme.example"

# Both are 'correct' but neither is guaranteed, and a downstream
# system parsing this output would need to handle both shapes.
```

04 Skills: packaging a repeatable procedure the model can auto-load

5:08

> A skill is a reusable bundle — a markdown file with metadata, optionally packaged with scripts and resources in a folder — that teaches the model to do a specific task the same way every time, loaded into context only when needed.

A skill packages a task the way you'd document a standard operating procedure: a title naming the skill, a description of when it should be used, and the actual instructions (the procedure or prompt) the model should follow when that situation comes up. This is typically stored as a markdown file with a small amount of metadata at the top, inside a folder that can also hold supporting scripts and reference resources the task needs.

The key behavioral property is that a skill is auto-loaded: the AI application inspects the description of each available skill against the current task and pulls the matching skill's full instructions into the context window only when it's relevant, rather than keeping every possible instruction set loaded all the time. A "debug this code" skill only enters the context when the user is actually asking about a code error; it costs nothing on unrelated requests. This keeps the context window focused, and it means you can maintain a whole library of task-specific procedures without every one of them competing for space in every request.

Because the instructions are fixed text pulled in verbatim, the model follows the same documented steps every time the skill is triggered, which is exactly the repeatability MCP alone can't provide. Skills work across major AI tools and models because the mechanism is simple — it's just structured text plus optional files, not a network protocol — which also makes them cheap to write, version, and share.

KEY POINTS

- → A skill has a name, a description of when to use it, and the instructions/prompt to run when triggered.
- → Skills are commonly stored as a markdown file with metadata, inside a folder that can also hold scripts and reference resources.
- → Skills are auto-loaded: the AI application matches the current task against each skill's description and only pulls in the matching one.
- → Because the instructions are literal, saved text, the model follows the same procedure every time the skill fires — this is how skills deliver repeatability.
- → Skills are lightweight compared to MCP: no server, no protocol, no authentication layer — just structured text and optional files.

A minimal skill file

A skill for formatting CRM customer summaries consistently. The frontmatter names the skill and tells the AI application when to load it; the body is the fixed procedure the model follows every time it's triggered.

```
---
name: crm-customer-summary
description: Use whenever the user asks for a customer summary from CRM data. Ensures a
consistent, repeatable output format.
---

# CRM Customer Summary

When given a customer record, always produce the summary in exactly
this structure, in this order:

1. Customer name
2. Primary contact email
3. Primary contact phone
4. Account owner
5. Any notes field, verbatim, or "None" if absent

Do not reorder or omit fields. Do not add commentary. Output as a
plain bullet list, one field per line.
```

05 Choosing MCP, skills, or both

5:53

> Reach for MCP when the agent needs controlled, permissioned access to real-time external data; reach for a skill when you need a lightweight, repeatable way to perform a task the model doesn't already do consistently.

MCP is the right tool when the agent's job depends on real, current state from an external system, and that access needs to be tightly permissioned. Questions like "what VMs am I currently running," "what's our cluster state," or "pull this customer's current record" all require live data and controlled, scoped access — exactly what an MCP server is built to provide. Think of MCP as an integration layer: the agent can call the tools and resources the server exposes, nothing more, which is also what keeps it safe to grant.

Setting up and maintaining an MCP server — running the server process, defining its tool schemas, managing authentication — is real infrastructure work. If all you need is a reusable, custom capability that doesn't require touching a live external system, standing up an MCP server for it is overkill. That's the situation skills are built for: they're lightweight, they teach the model how to do something (fetch investment data and analyze it a certain way, run a specific compliance checklist, always format a report the same way), and they can bundle their own scripts and examples without needing a running service behind them.

The two are not mutually exclusive. A well-built agent commonly uses both: MCP to reach into a CRM or database for live, permissioned data, and a skill to guarantee that whatever comes back gets processed and formatted the exact same way every time. Both mechanisms exist to enhance what's in the model's context window at the moment it needs to answer, and both are open, widely adopted across AI tools, and usable locally without waiting on a vendor.

KEY POINTS

- → Use MCP when the task needs live, current data from an external system under tight, scoped permissions.
- → MCP access is call-only: the agent can invoke the tools and resources the server exposes, and nothing beyond that.
- → Use a skill when you need a lightweight, reusable, repeatable capability and don't need to reach a live external system.
- → Standing up an MCP server is real infrastructure (process, schemas, auth) — overkill for a task that's really just "do this the same way every time."
- → MCP and skills are commonly combined: MCP fetches the live data, a skill enforces how it's processed and formatted.

One agent, both mechanisms

An agent that answers "summarize our top customer's account" can use an MCP tool call to fetch the live record, then apply a skill so the summary it produces is always shaped the same way.

```
# Step 1 — MCP: fetch live, permissioned data
tool_call = {
    "name": "get_customer",
    "arguments": {"customer_name": "Acme Corp"}
}
# -> MCP server authenticates with its own scoped token, calls the
#    CRM's real API, returns the current record into the context window.

# Step 2 — Skill: enforce a repeatable output procedure
# The 'crm-customer-summary' skill's instructions are auto-loaded
# because the request matches its description, and the model formats
# the fetched record using that fixed 5-field structure every time.
```

Glossary

Context window

The span of text an LLM can see at once when generating a response, including the prompt, any supplied data, and prior conversation.

Prompt engineering

Crafting the wording, role, and task of a request to an LLM to shape its response.

Context engineering

Deciding what information — data, formatting rules, tool output — needs to be present in the model's context window for it to answer correctly, beyond just the wording of the prompt.

MCP (Model Context Protocol)

A standardized protocol that lets an AI application call external services and data sources through a common, LLM-readable interface, handling both request translation and authentication.

MCP server

A server that implements MCP for a specific service, exposing a set of typed tools to the AI application and translating tool calls into that service's real API requests.

Scoped token

An access credential limited to exactly the permissions (e.g. read-only, or write access to one resource type) that a given task needs, rather than a broad, all-access credential.

Skill

A reusable, packaged set of instructions — typically a markdown file with metadata, optionally bundled with scripts and resources — that teaches an LLM to perform a specific task the same way every time, loaded into context only when relevant.

Non-deterministic

Producing different outputs on different runs even given the same input, which is a property of how LLMs generate text and why unenforced instructions don't guarantee repeatable results.

Auto-loading (of a skill)

The behavior where an AI application matches the current task against each available skill's description and pulls only the matching skill's instructions into the context window.

JSON-RPC

A lightweight remote-procedure-call protocol encoded in JSON, used by MCP to structure requests like tools/list and tools/call between client and server.

Check yourself

Answer first, then open each one.

Why is context engineering a bigger idea than prompt engineering?

Prompt engineering only shapes how a question is worded and framed; it doesn't give the model any information it lacks. Context engineering is about deciding what data, rules, or tool output need to be placed in the context window so the model has what it actually needs to answer correctly — the prompt is just one part of that.

Why is pasting an API's raw documentation and a bare access token into an LLM's context a worse approach than using an MCP server for that same API?

The model would have to correctly interpret arbitrary documentation and generate valid raw HTTP requests purely through free-form text generation, with a broad credential and no structural guardrails. An MCP server instead exposes a small set of typed, well-defined tools and holds a uniquely scoped token itself, so the model only has to emit a structured request for a named tool — far less room for malformed calls or over-broad access.

A team has MCP wired up to their CRM and complains that summaries of the same customer record still come out differently formatted each time. What's actually missing, and why doesn't a better-worded prompt fix it permanently?

MCP only solves reliable access to the data; it says nothing about how the model processes or formats that data afterward. LLMs are non-deterministic, so instructions repeated in a prompt are advisory and can still be interpreted differently across runs. What's missing is a skill: a fixed, auto-loaded procedure that pins the output structure every time the task recurs, rather than relying on the wording of each individual prompt.

Why would setting up an MCP server be overkill for something like 'always format Excel cleanup output the same way,' when a skill would not be?

That task doesn't require reaching a live external system or enforcing scoped, permissioned access to one — it just needs a fixed, repeatable procedure. An MCP server means standing up a running process, defining tool schemas, and managing authentication, which is infrastructure this task doesn't need. A skill is just structured instructions (plus optional scripts) that load when relevant, with none of that overhead.

Why does it make sense for a single agent to use both MCP and a skill on the same task, rather than treating them as alternatives?

They solve different problems: MCP provides controlled, permissioned access to live external data, while a skill provides a repeatable procedure for how that data (or any task) gets processed once it's in context. An agent that needs both live data and a guaranteed-consistent output format needs MCP to fetch the data and a skill to enforce how it's handled — neither one alone covers both requirements.

Where to go next

- → Read the Model Context Protocol specification and inspect the tools/list and tools/call JSON-RPC message shapes for a real MCP server.
- → Write a minimal skill file (name, description, instructions) for a repetitive task you do often with an LLM, and observe whether it gets auto-loaded only when relevant.
- → Build a small MCP server that exposes one tool against a mock API, and trace how a scoped token limits what the agent can do compared to a raw API key.
- → Compare a task done with a loosely worded prompt across five runs against the same task done with a skill, and note how much the output format varies in each case.
- → Look into using CLIs as an integration layer for building agents, an approach the video flags as a separate topic worth its own treatment.
